样题1

项目一：平台搭建与运维

(一)任务一：大数据平台搭建

1.子任务一：基础环境准备

(1)`hostnamectl set-hostname master` 配置主机名
`ifconfig` 查看ip
`vim /etc/hosts`

```
ip master
ip slave1
ip slave2
```

(2)`ping slave1 -c 1 && ping slave2 -c 1`
(3)`ssh-keygen -t rsa -b 4096` 生成密钥
`ssh-copy-id root@slave1` 复制密钥到其他节点
`ssh-copy-id root@slave2`
`sudo systemctl restart sshd` 重启ssh服务
`ssh slave1(slave2)` ssh连接
(4)`tar -zxvf /opt/software/jdk-8u391-linuxx64.tar.gz -C /opt/module`
`ll /opt/module`
(5)`vim /etc/profile`

```
export JAVA_HOME=/opt/module/jdk
export PATH=$PATH:$JAVA_HOME/bin
```

`source /etc/profile`
(6)`java -version` 查看Java版本
(7)`useradd hadoop`
`passwd hadoop`
`visudo /etc/sudoers` 配置管理员

```
hadoop ALL=(ALL)    NOPASSWD:ALL
```

`grep 'hadoop' /etc/sudoers`
(8)`systemctl stop firewalld` 关闭防火墙

`systemctl disable firewalld` 开机不自启
`systemctl status firewalld`

2.子任务二：Hadoop 完全分布式安装配置

(1)`tar -zxvf /opt/software/hadoop-3.1.3.tar.gz -C /opt/module`
(2)`mv /opt/module/hadoop-3.1.3 /opt/module/hadoop` 重命名
`chown -R hadoop:hadoop /opt/module/hadoop` 修改所属主/组
(3)`cd /opt/module/hadoop/etc/hadoop`
hadoop-env.sh

```shell
export JAVA_HOME=/opt/module/jdk
export HADOOP_HOME=/opt/module/hadoop
export HADOOP_CONF_DIR=${HADOOP_HOME}/etc/hadoop
export HADOOP_LOG_DIR=${HADOOP_HOME}/logs
```

core-site.xml

```xml
<property>
   <name>fs.defaultFS</name>
   <value>hdfs://master:9820</value>
</property>
<property>
   <name>hadoop.tmp.dir</name>
   <value>/opt/module/hadoop/tmp</value>
</property>
```

hdfs-site.xml

```xml
<property>
   <name>dfs.replication</name>
   <value>3</value>
</property>
<property>
   <name>dfs.namenode.secondary.http-address</name>
   <value>master:9896</value>
</property>
<property>
   <name>dfs.datanode.data.dir</name>
   <value>/opt/module/hadoop/datanode/data</value>
</property>
```

mapred-site.xml

```xml
<property>
   <name>mapreduce.framework.name</name>
   <value>yarn</value>
</property>
<property>
   <name>mapreduce.jobhistory.address</name>
   <value>master:10020</value>
</property>
<property>
   <name>mapreduce.jobhistory.webapp.address</name>
   <value>master:19888</value>
</property>
```

yarn-site.xml

```xml
<property>
   <name>yarn.resourcemanager.hostname</name>
   <value>master</value>
</property>
<property>
   <name>yarn.nodemanager.aux-services</name>
   <value>mapreduce_shuffle</value>
</property>
<property>
   <name>yarn.log-aggregation-enable</name>
   <value>true</value>
</property>
<property>
   <name>yarn.log.server.url</name>
   <value>http://master:19888/jobhistory/logs</value>
</property>
```

works

```
master
slave1
slave2
```

(4)`scp -r /opt/module/hadoop hadoop@slave1:/opt/module`
`scp -r /opt/module/hadoop hadoop@slave2:/opt/module`
`ssh slave1(slave2)`
`chown -R hadoop:hadoop /opt/module/hadoop` 其余两个节点hadoop 安装目录的所有文件修改所属组者
(6)`vim /etc/profile`

```
export HADOOP_HOME=/opt/module/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

`source /etc/profile`

(7)`hdfs namenode -format` 初始化Hadoop环境namenode

(8)`start-dfs.sh`

`start-yarn.sh`

`mapred --daemon start historyserver`

`jps`

3.子任务三：Zookeeper 集群安装配置

(1)`tar -zxvf /opt/software/apache-zookeeper-3.5.7-bin.tar.gz -C /opt/module`

(2)`mv /opt/module/apache-zookeeper-3.5.7-bin.tar.gz /opt/module/zookeeper`

(3)`vim /etc/profile`

```
export ZOOKEEPER_HOME=/opt/module/zookeeper
export PATH=$PATH:$ZOOKEEPER_HOME/bin
```

`source /etc/profile`

(4)`cd /opt/module/zookeeper/conf`

`mv zoo_simple.cfg zoo.cfg`

`vim zoo.cfg`

```
server.1=master:2888:3888
server.2=slave1:2888:3888
server.3=slave2:2888:3888
```

(5)`cd /opt/module/zookeeper`

`mkdir data`

`vim myid` 并将文件内容设置为1

(6)`scp -r /opt/module/zookeeper root@slave1:/opt/module`

`scp -r /etc/profile root@slave1:/etc/profile`

(7)`ssh slave1`

`vim myid` 文件内容修改为2

(8)`zkServer.sh start`

(9)`zkServer.sh status`

(10)`jps`

4.子任务四：Kafka 安装配置

(1)`tar -zxvf /opt/software/kafka_2.12-2.4.1.tgz -C /opt/module`
`mv /opt/software/kafka_2.12-2.4.1 /opt/moudle/kafka`
(2)子任务三第四题
(3)`vim /etc/profile`

```
export KAFKA_HOME=/opt/module/kafka
export PATH=$PATH:$KAFKA_HOME/bin
```

(4)`kafka-server-start.sh --version`
(5)`cd /opt/module/kafka/config`
`vim server.properties`

```
broker.id=1
zookeeper.connect=master:2181,slave1:2181,slave2:2181/kafka
```

`scp -r /opt/module/kafka root@slave1:/opt/module`
`scp -r /opt/module/kafka root@slave2:/opt/module`
(6)`kafka-topics.sh --bootstrap-server master:9092 --create --topic installtopic --partitions 2 --replication-factor 2`

(二)任务二：数据库服务器的安装与运维

1.子任务一：MySQL 安装配置

(1)`tar -zxvf /opt/software/mysql-5.7.25-1.el7.x86_64.rpm-bundle.tar -C /opt/module`
`rpm -ivh mysql-communitycommon mysql-community-libs mysql-community-libs-compat mysql-community-client mysql-community-server`
(3)`systemctl start mysqld`
`mysqld --initialize --user=mysql`

2.子任务二：MySQL 运维

(1)`mysql -version`
(2)`create database tree_statistics default character set utf8mb4;`
(3)`create user 'tree_admin'@'%' identified by '密码'`
`grant all privileges on tree_statistics.* to 'tree_admin'@'%';`
`create user 'tree_viewer'@'%' identified by '密码'`
`grant select on tree_statistics.* to 'tree_viewer'@'%';`
`flush privileges;` 刷新权限
(4)

```shell
cat > /../backup.sh << 'EOF'
#!/bin/bash
user="root"
passwd="zjm5404"
backup='/../..'
date=$(date +%Y%m%d_%H%M%S)
database="tree_statistics"

mkdir -p "$backup"

mysqldump -u"$user" -p"$passwd" "$database" > "${backup}/${databse}_backup_$date.sql"

find "$backup" -name "${database}_backup_*.sql" -mtime +7 -delete
EOF
```

`chomd +x /../backup.sh`
`(crontab -l 2>/dev/null; echo "0 1 * * * /.../backup.sh") | crontab -`
`crontab -l`
(5)`show variables like 'max_connections';`
`show variables like 'query_cache_size';`
(6)`vim /etc/my.cnf`

```
max_connections = 800
query_cache_size = 128M
```

(7)`use tree_statistics`
`show engines;`
(8)`create user 'backup_operator'@'%' identifity by '密码'`
`grant bakup_admin,reload on tree_statistics.* to 'backup_operator'@'%'`

3.子任务三：数据表的创建及维护

`use tree_statistics;`
(1)

```
create table tree_info(
   tree_id int primary key,
   species varchar(100) not null,
   region varchar(50) not null,
   age int not null,
   height decimal(6,2) not null,
   diameter decimal(6,2),
   health_status enum('健康','一般','较差') default '健康',
   planting_date date
);
```

(2)

```
create table region_info(
   region_id int primary key,
   region_name varchar(50) not null,
   area_size decimal(10,2),
   soil_type varchar(50),
   manager varchar(100)
);
```

(3)

```
create table growth_records(
   record_id int primary key,
   tree_id int,
   measure_date date not null,
   new_height decimal(6,2),
   health_update enum('健康','一般','较差'),
   note varchar(200),
   foreign key (tree_id) references tree_info(tree_id)
);
```

(4)统计每个区域的树木数量和平均树龄

```
select count(tree_id) as 树木数量,region as 所在区域, avg(age) as 平均年龄
from tree_info
group by region;
```

查询高度高于该区域平均高度的树木信息

```
select
t1.tree_id,t1.species,t1.region,t1.age,t1.height,t1.diameter,t1.health_status,t1.plan
ting_date
from tree_info as t1
where (
   select avg(t2.height) from tree_info as t2
   where t2.region = t1.region
);
```

查询主要土壤类型为“壤土”的区域及其树木数量

```
select ri.region_name,count(tree_id) as 树木数量
from region_info as ri
join tree_info as ti
on ri.region_name = ti.region
where ri.soil_type = "壤土"
group by  ri.region_name;
```

统计每种树种的最高高度和平均胸径，并按最高高度降序排列

```
select t.species,max(t.height) as 最高高度,avg(t.diameter) as 平均胸径
from tree_info as t
group by t.species
order by 最高高度 desc;
```

查询最近一年内有生长记录的树木信息及其所在区域详情

```
select t.*,r.*g.*
from growth_records as g
join tree_info as t
on ti.tree_id = gr.tree_id
join region_info as r
on r.region_name = t.region
where timestampdiff(year,t.planting_date,g.measure_date) <= 1;
```

项目二：数据获取与处理

(一)任务一：数据获取与清洗

1.子任务一：数据获取

有一份古树名木调查数据，包含区域、地块名称、树种、生长状况、树龄段、养护情况、是否古树名木、胸径(cm)、估值(元)、调查年份等信息，数据已存入到 `tree_survey.csv` 文件中，请使用 pandas 读取 `tree_survey.csv` 并将数据集的前 10 行打印在 IDE 终端

```
import pandas as pd
data = pd.read_csv('tree_survey.csv',nrows=10)
print(data)
```

2.子任务二：使用 Python 进行数据清洗

(1)删除胸径(cm)为空或为零的记录，存储为 `cleaned_tree_c1_17.csv`（N=17）
(2)删除估值(元)为空或异常高（超过平均估值的 3 倍）的记录，存储为 `cleaned_tree_c2_10.csv`（N=10）
(3)对树种数据进行标准化，将「法桐」「悬铃木」统一转换为标准名称「二球悬铃木」，存储为 `cleaned_tree_c3_23.csv`（N=23）
(4)删除是否古树名木字段为空白的记录，存储为 `cleaned_tree_c4_5.csv`（N=5）
(5)删除胸径(cm)小于 20（树木过小，不符合统计标准）的记录，存储为 `cleaned_tree_c5_7.csv`（N=7）

(二)任务二：数据标注

1.子任务一：价格区间标注

按「区域」分组计算该区域平均估值，将每条记录与所在区域的平均估值比较：低于 70% 记为「低价值」、在 70%～130% 之间记为「中价值」、超过 130% 记为「高价值」，结果存入 `value_range_mark.csv`（161 行，列：编号、地块名称、胸径(cm)、估值(元)、价值区间）。标注结果分布为：中价值 63 条（39.13%）、低价值 50 条（31.06%）、高价值 48 条（29.81%），三个价值区间的记录分布相对均衡。

```
编号 地块名称 胸径(cm) 估值(元) 价值区间
1 福田区-滨河绿地 21.61 1565.0 低价值
2 福田区-深圳湾公园 86.57 4381.0 高价值
3 罗湖区-滨河绿地 36.22 1583.0 低价值
```

2.子任务二：市区热门度标注

按「区域」统计树木记录数量，以全市平均数量 32.2 条为基准：高于 120%（38.64 条）记为「高丰富度」、在 80%～120%（25.76～38.64 条）之间记为「中丰富度」、低于

80%（25.76 条）记为「低丰富度」，结果存入 `area_richness_mark.csv`（161 行，列：编号、地块名称、区域、胸径(cm)、估值(元)、区域丰富度）。标注结果分布为：中丰富度 118 条（73.29%）、高丰富度 43 条（26.71%），其中龙岗区（43 条）为高丰富度，福田区（33 条）、宝安区（29 条）、罗湖区（28 条）、南山区（28 条）均为中丰富度，本数据集无低丰富度区域。

```
编号 地块名称 区域 胸径(cm) 估值(元) 区域丰富度
1 福田区-滨河绿地 福田区 21.61 1565.0 中丰富度
2 福田区-深圳湾公园 福田区 86.57 4381.0 中丰富度
3 罗湖区-滨河绿地 罗湖区 36.22 1583.0 中丰富度
```

(三)任务三：数据统计

1.子任务一：HDFS 文件上传下载

(1)`hdfs dfs -mkdir /forestry_data` 在 HDFS 目录下新建目录 /forestry_data
(2)`hdfs dfs -chmod 777 /forestry_data` 修改权限，赋予目录 /forestry_data 最高 777 权限
(3)`hdfs dfs -get /forestry_data /tmp` 下载 HDFS 新建目录 /forestry_data 到本地容器 master 指定目录 /tmp 下

2.子任务二：计算输入文件中的单词数

(1)`hdfs dfs -mkdir -p /user/forester/input` 在 HDFS 上创建 /user/forester/input 目录
(2)`hdfs dfs -put /var/log/tree_species_list.txt /user/forester/input` 在 master 节点将本地树木调查报告文件上传至该目录
(3)`hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar wordcount /user/forester/input/tree_species_list.txt /user/forester/output` 使用 Hadoop 提供的 wordcount 示例对 tree_species_list.txt 做单词（视为树种名称或统计单元）统计，结果存储到 /user/forester/output 目录
(4)`hdfs dfs -cat /user/forester/output/part-r-00000 | head -n 10` 查看统计结果并将结果前十行截图

3.子任务三：使用拟蒙特卡罗法估算 Pi 值

(1)`cd /opt/module/hadoop/share/hadoop/mapreduce`
`hadoop jar hadoop-mapreduce-examples-3.1.3.jar pi 16 10000` 通过 Hadoop 提供的 Pi 示例，使用 16 个映射（每个映射 10,000 个示例）来估算 pi 值
(2)Pi 值估算结果如下（16 个 map，每个 map 处理 10000 个样本，总样本数 160000）

```
Estimated value of Pi is 3.1415xxxxx
```

估算公式为 `pi = 4 × 单位圆内点数 / 总样本数`，理论期望值 `pi ≈ 3.14159265358979`，实际运行结果在该值附近波动。

项目三：业务分析与可视化

(一)任务一：数据分析与可视化

1.子任务一：数据分析

(1)不同区域的树木调查数量倒序前三名：龙岗区 43 条、福田区 33 条、宝安区 29 条（其后为罗湖区 28 条、南山区 28 条）
(2)各类树种的平均估值倒序前三种：荷花玉兰 4017.10 元、二球悬铃木 3750.09 元、木棉 3710.13 元
(3)各区域的平均估值倒序前三名：福田区 3352.61 元、罗湖区 3289.93 元、龙岗区 3237.26 元
(4)不同生长状况的树木数量正序展示：健康 43 条、较差 54 条、一般 64 条
(5)估值低于平均估值 3227.64 元但胸径超过平均胸径 63.89 cm 的树木记录数量为 6 条，这类树木估值不高但胸径较大，属于生长潜力未被充分定价的对象

2.子任务二：数据可视化

(1)使用柱状图展示各区域的树木调查数量，柱状图中的每个柱子代表一个区域，高度代表该区域的树木记录数量
(2)创建条形图比较不同树种的平均估值（元），条形图中横轴表示平均估值，纵轴表示树种
(3)使用折线图展示不同调查年份的树木平均估值趋势
(4)制作散点图探索树木估值（元）与胸径（cm）之间的关系，散点图中的横轴为估值，纵轴为胸径，每个点代表一株被调查的树木

(二)任务二：业务分析

(1)平均估值：数据集中所有树木记录的平均估值为 3227.64 元
(2)最优势树种：二球悬铃木的调查数量最多，共 32 条，分布最广，是全市占比最高的优势树种
(3)高丰富度区域特征：记录数量在前 25% 的区域为龙岗区和福田区（门槛为 33.0 条），其共有特征如下
(1)估值范围在 775.00 ~ 6277.00 元，平均估值 3287.34 元，高于全市平均估值 3227.64 元
(2)胸径范围在 20.85(cm) ~ 119.71(cm)，平均胸径 64.39(cm)，略高于全市平均水平
(3)主要生长状况为较差，说明这些区域的存量树木以大胸径老树为主，生长状况亟需跟踪
1.城市林木资源管理与生态规划策略建议：根据上述分析结果，提出以下策略建议
(1)聚焦优势树种养护：二球悬铃木是调查数量最多的树种（共 32 条），分布最广、生态与景观作用最大，应优先保障其水肥、修剪与病虫害防治投入，并以其为主要基调树种纳入城市绿地系统规划
(2)按区域分级管理：记录数量在前 25% 的区域为龙岗区、福田区，其平均估值约 3287.34

元、平均胸径约 64.39 cm，主要生长状况为较差，这些区域应作为重点养护片区，实行定期普查与动态建档，防止高价值林木流失
(3)建立价值分层保护机制：全市平均估值为 3227.64 元，对估值低于均价但胸径超过平均水平的 6 条林木，应重点核查其树龄与古树名木潜力，符合标准的及时挂牌保护，避免因估值偏低在建设中遭到砍伐或移植

项目四：AI 大模型应用开发综合实践

(一)任务一：AI 大模型服务综合部署

1.子任务一：基本配置

(1)`mkdir -p /media/cdrom`
情况一：镜像文件在 `/opt/software` 目录下

```
mount -o loop /opt/software/openEuler-22.03-LTS-SP4-x86_64-dvd.iso /media/cdrom
```

情况二：镜像已通过虚拟光驱加载到设备 `/dev/sr0`

```
mount /dev/sr0 /media/cdrom
```

编辑 `vim /etc/fstab`

```
末尾添加
/dev/sr0    /media/cdrom    iso9660    defaults,ro    0    0
```

注意：
`/dev/sr0` 光驱设备名（用 `lsblk` 确认）
`/media/cdrom` 挂载点
`iso9660` 光盘文件系统类型
`defaults,ro` 表示默认选项且只读（推荐加 `ro` 以避免警告）
`0 0` 表示不备份、不检查
`mount -a` 验证
如果没有报错，说明配置成功
(2)`vim /etc/yum.repos.d/local.repo` 软件源配置文件的编写

```
[local]
name=Local Repository
baseurl=file:///media/cdrom
enabled=1
gpgcheck=0
```

`yum makecache` 建立缓存

```
(可能需要删除所有默认远程源)
cd /etc/yum.repos.d/
ls 查看是否有默认远程源
rm -f /etc/yum.repos.d/openEuler.repo 删除除了local.repo以外的
```

验证是否仅使用本地源

2.子任务二：环境搭建

`useradd ollama`
`groupadd ollama`

(二)任务二：AI 大模型综合应用开发

1.子任务一：Dify 运行环境搭建

本任务需要使用 root 用户完成相关配置，已提供安装 dify 及需要配置的前置环境
(1)`yum install -y docker` 在 AI-SRV 中完成 Docker 环境的安装
`systemctl start docker`
`systemctl enable docker`
`docker --version` 查看 Docker 版本信息
(2)`tar -zxvf /root/dify-0.14.2.tar.gz -C /opt` 将 dify-0.14.2.tar.gz 解压至 /opt 路径中
`cd /opt/dify/docker` 进入 dify 的 docker 目录
(3)`docker load -i /opt/dify/docker/dify-images/*.tar` 完成 dify 镜像的导入，相关文件位于 dify-images 目录中
`docker images` 检查镜像列表
(4)`docker compose up -d` 完成 dify 的启动，并在客户机中访问 AI-SRV 中的 dify web 服务
`docker ps` 查看容器运行状态
(5)在客户机中访问 dify，完成用户注册，邮箱为 admin@dify.ai，用户名为 admin，密码为 Qwer1234，注册后登录 dify 管理页面
(6)在客户机中访问 dify，完成对 ollama 模型提供商的注册，API 地址填 `http://<AI-SRV 的 IP>:11434`，模型名称选已导入的 qwen2.5:1.5b

2.子任务二：林业数据智能助手构建

(1)根据林业数据分析场景需要，创建空白应用 "Forestry Data Assistant"
(2)为用户提供林业数据查询功能，其中参数包含 Query（查询内容），支持以下类型的查询
树种信息查询（如："查询"香樟"的平均估值、分布区域"）
区域统计查询（如："查询"南山区"的树木总数、平均胸径"）
生长状况分析（如："查询健康状况为"较差"的树木分布情况"）
价值评估查询（如："查询估值超过 10000 元的古树信息"）
(3)为确保结果准确性，需要从以下多个维度提供数据支持
基于 MySQL 数据库中的结构化数据（tree_statistics 数据库）
基于清洗后的 CSV 文件（tree_survey.csv 及相关处理结果）
基于大模型的智能解析和推理能力
(4)配置提示词要求
系统角色设定为"林业数据分析专家"
支持 SQL 查询语句的自动生成和解释
支持自然语言到数据分析需求的转换
输出结果需包含数据来源说明
(5)运行应用，测试以下查询场景
"查询所有古树名木的平均胸径和最大估值"
"南山区哪种树种的分布数量最多？"
"健康状况较差的树木主要分布在哪些区域？"
"对比壤土区域和其他土壤类型的树木平均高度"